

UNIVERSIDADE ESTADUAL DA PARAÍBA – UEPB

Campus I

Centro de Ciência e Tecnologia – CCT

Departamento de Computação

Bacharelado em Ciência da Computação

GUSTAVO AZEVEDO LELIS FARIAS

**PROJETO EXPERIMENTAL COMPARATIVO DE TÉCNICAS DE PROJETO DE
ALGORITMOS**

Relatório submetido ao Curso de Ciência da Computação da Universidade Estadual da Paraíba, na disciplina de Técnicas de Análise de Algoritmos, correspondente ao período letivo de 2026.1.

Campina Grande – PB

2026

RESUMO

Este relatório apresenta a implementação e a análise comparativa de duas técnicas exatas de busca no espaço de soluções, Backtracking e Branch and Bound, aplicadas aos problemas de Partição de Conjuntos e Árvore Geradora Mínima (AGM). O estudo foi conduzido com implementação em Java, com instrumentação de tempo de execução, uso de memória, nós visitados e nós podados. Os resultados mostram que ambas as técnicas mantêm corretude nas instâncias avaliadas, com vantagem prática do Branch and Bound em parte dos cenários, a depender da qualidade dos limites utilizados.

SUMÁRIO

- 1 INTRODUÇÃO
 - 1.1 Contextualização do problema
 - 1.2 Objetivos
 - 1.2.1 Objetivo geral
 - 1.2.2 Objetivos específicos
- 2 METODOLOGIA E IMPLEMENTAÇÃO
 - 2.1 Estrutura do sistema desenvolvido
 - 2.2 Formato de dados de entrada e saída
 - 2.3 Métricas de avaliação
- 3 ALGORITMOS IMPLEMENTADOS
 - 3.1 Backtracking
 - 3.2 Branch and Bound
- 4 EXPERIMENTOS E RESULTADOS
 - 4.1 Resultados para Partição de Conjuntos
 - 4.2 Resultados para AGM
- 5 ANÁLISE E DISCUSSÃO
- 6 CONCLUSÃO
- 7 REFERÊNCIAS

1 INTRODUÇÃO

1.1 Contextualização do problema

Este trabalho apresenta uma comparação entre duas técnicas clássicas de busca exata: Backtracking e Branch and Bound. A análise foi aplicada a dois problemas: Partição de Conjuntos e Árvore Geradora Mínima (AGM). A escolha desses problemas permite discutir corretude, custo computacional e comportamento de poda em contextos de busca combinatória e em grafos.

1.2 Objetivos

1.2.1 *Objetivo geral*

Implementar e comparar formalmente as estratégias de Backtracking e Branch and Bound em uma primeira entrega do projeto.

1.2.2 *Objetivos específicos*

- Analisar a complexidade assintótica teórica de cada técnica.
- Avaliar tempo de execução e uso de memória.
- Verificar corretude das soluções obtidas.
- Investigar escalabilidade em função do tamanho da entrada.
- Quantificar nós visitados e nós podados.
- Aplicar as técnicas aos problemas de Partição de Conjuntos e AGM, respeitando os formatos de entrada e saída.

2 METODOLOGIA E IMPLEMENTAÇÃO

2.1 Estrutura do sistema desenvolvido

Os experimentos foram realizados em Java. A medição de tempo foi feita com instrumentação interna de cada algoritmo, e o uso de memória foi registrado por métricas de execução da JVM. As implementações foram organizadas nos arquivos:

- PartitionSolver.java
- MSTSolver.java
- Main.java
- BenchmarkMain.java

2.2 Formato de dados de entrada e saída

Problema de Partição de Conjuntos

Entrada:

```
n
a1 a2 a3 ... an
```

Saída:

```
YES
k
subset1
subset2

Ou: NO
```

Problema de Árvore Geradora Mínima (AGM)

Entrada:

```
n m
u1 v1 w1
u2 v2 w2
...
um vm wm
```

Saída:

```
peso_total
u1 v1 w1
u2 v2 w2
...
u(n-1) v(n-1) w(n-1)
```

2.3 Métricas de avaliação

As técnicas foram comparadas por:

- Tempo de execução (ms)
- Uso de memória (KB)
- Quantidade de nós visitados
- Quantidade de nós podados
- Corretude da solução encontrada

3 ALGORITMOS IMPLEMENTADOS

3.1 Backtracking

Backtracking explora o espaço de soluções de forma sistemática, com retrocesso quando uma solução parcial viola restrições. No pior caso, apresenta comportamento exponencial em problemas combinatórios, mas pode ter ganhos práticos com podas de viabilidade.

Fragmento de implementação (Backtracking – Partição de Conjuntos):

```
private void btRecursive(int idx, int currentSum,
    List<Integer> currentIndices, PartitionData result) {
    nodesVisited++;
    if (result.found) return;
    if (currentSum > targetSum) {
        nodesPruned++;
        return;
    }
    if (currentSum + elements[idx] <= targetSum) {
        currentIndices.add(idx);
        btRecursive(idx + 1, currentSum + elements[idx],
            currentIndices, result);
        currentIndices.remove(currentIndices.size() - 1);
    }
    btRecursive(idx + 1, currentSum, currentIndices, result);
}
```

O trecho mostra a exploração recursiva com poda por inviabilidade (soma acima do alvo), mantendo a corretude da busca exata.

3.2 Branch and Bound

Branch and Bound também preserva corretude, porém adiciona limites para descartar ramos incapazes de melhorar a melhor solução corrente. Em geral, o desempenho prático depende da qualidade desses limites.

Fragmento de implementação (Branch and Bound – AGM):

```
while (!queue.isEmpty()) {
    Node node = queue.poll();
    nodesVisited++;
    if (node.edges.size() == nVertices - 1) {
        if (node.weight < bestWeight) {
            bestWeight = node.weight;
            bestEdges = new ArrayList<>(node.edges);
        }
        continue;
    }
    if (node.weight >= bestWeight) {
        nodesPruned++;
        continue;
    }
}
```

Nesse trecho, a regra `node.weight >= bestWeight` implementa o bound de poda: nós que não podem melhorar a melhor solução corrente são descartados.

4 EXPERIMENTOS E RESULTADOS

4.1 Resultados para Partição de Conjuntos

Instância de referência: `elements = [1, 5, 11, 5]`.

Tabela 1 – Comparação de desempenho na Partição de Conjuntos

Técnica	Encontrou partição	Tempo (ms)	Memória (KB)	Nós visitados	Nós podados
Backtracking	Sim	0,6326	0,00	8	0
Branch & Bound	Sim	1,2838	0,00	18	6

Fonte: elaborado pelo autor (2026), com base na execução de BenchmarkMain.

Tabela 2 – Solução encontrada para Partição (técnicas exatas)

Técnica	Subconjunto 1	Subconjunto 2	Soma(subset1)	Soma(subset2)
Backtracking	[1, 5, 5]	[11]	11	11
Branch & Bound	[11]	[1, 5, 5]	11	11

Fonte: elaborado pelo autor (2026).

4.2 Resultados para AGM

Instância de referência: 5 vértices e 7 arestas.

Tabela 3 – Comparação de desempenho na AGM

Técnica	Peso da AGM	Tempo (ms)	Memória (KB)	Nós visitados	Nós podados
Backtracking	10	1,6533	0,00	145	16
Branch & Bound	10	1,2538	0,00	163	36

Fonte: elaborado pelo autor (2026), com base na execução de BenchmarkMain.

Tabela 4 – Arestas de uma AGM válida (instância de referência)

u	v	w
1	2	1
0	2	2
3	4	2
1	3	5

Fonte: elaborado pelo autor (2026).

5 ANÁLISE E DISCUSSÃO

Tabela 5 – Síntese comparativa das técnicas na primeira entrega

Critério	Backtracking	Branch & Bound
Corretude em Partição	Exata	Exata
Corretude em AGM	Exata (instâncias peq.)	Exata (instâncias peq.)
Custo teórico (pior caso)	Exponencial	Exponencial
Facilidade de implementação	Alta	Média
Escalabilidade prática	Baixa a média	Média

Fonte: elaborado pelo autor (2026).

Os resultados mostram que ambas as técnicas são corretas para as instâncias avaliadas. Em Partição, Backtracking apresentou menor tempo nesta execução específica. Em AGM, Branch and Bound obteve menor tempo. Como se trata de medição empírica, os valores podem variar entre execuções por carga de CPU, estado da JVM e condições do sistema.

6 CONCLUSÃO

A primeira entrega confirmou a viabilidade de Backtracking e Branch and Bound para os problemas selecionados, com correteza preservada nas instâncias analisadas. A comparação evidenciou que o desempenho prático depende do problema e da efetividade das podas/limites. Como continuidade, recomenda-se ampliar a bateria de testes e incluir análise estatística com repetições para consolidar a discussão de escalabilidade.

7 REFERÊNCIAS

- CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Introduction to Algorithms. 3. ed. Cambridge: MIT Press, 2009.
- HOROWITZ, Ellis; SAHNI, Sartaj; RAJASEKARAN, Sanguthevar. Fundamentals of Computer Algorithms. 2. ed. Hoboken: Silicon Press, 2007.
- SKIENA, Steven S. The Algorithm Design Manual. 2. ed. New York: Springer, 2008.